

MOBILE APPLICATION SECURITY

DIVA ANDROID SECURITY ASSESSMENT

Static and dynamic assessment of a deliberately vulnerable Android application. Twelve exercises confirmed across insecure storage, injection, exported components, unsafe URI handling, native secrets, and Frida runtime instrumentation.

ANDROID 11

JADX

ADB

FRIDA

OWASP MASVS

EVIDENCE-LED MOBILE TESTING

This report records the work completed in an isolated Genymotion Android lab against DIVA (Damn Insecure and Vulnerable App). Every result was established through a repeatable sequence: inspect the implementation, form a hypothesis, validate it on the device, preserve evidence, and document the remediation.

12

CHALLENGES CONFIRMED

4

ANALYSIS TOOLS

5

MASVS AREAS

1

AUTHORIZED LAB

SCOPE

Local DIVA target on a rooted Android 11 x86_64 emulator. Testing used JADX, ADB, Android component tooling, native string extraction, and Frida.

PUBLIC-SAFETY HANDLING

This PDF preserves technical proof while redacting seeded lab credentials and extracted training values. The target APK and decompiled third-party source are excluded.

Accuracy note. DIVA has 13 exercises. This report covers Challenges 1–12. Challenge 13, a native memory-corruption / denial-of-service exercise, remains outside this completed scope.

ASSESSMENT WORKFLOW

PHASE	WHAT WAS DONE	PROOF RETAINED
Static	Reviewed decompiled Java, Android resources, manifest declarations, and native library strings.	Source paths and extracted indicators.
Dynamic	Triggered flows in DIVA and used ADB, Android component tools, SQLite, and Frida.	Command results, UI confirmations, and hook output.
Reporting	Mapped impact and remediation to mobile security practice.	Sanitized evidence register and this report.

TWELVE CONFIRMED EXERCISES

#	EXERCISE	VALIDATED RESULT	MASVS
1	Insecure Logging	Submitted payment data reached Logcat.	STORAGE
2	Hardcoding 1	Java-embedded vendor key recovered and accepted.	CRYPTO
3	Storage 1	Plaintext credentials recovered from SharedPreferences.	STORAGE
4	Storage 2	Plaintext credential row recovered from SQLite.	STORAGE
5	Storage 3	Plaintext record recovered from an app-private file.	STORAGE
6	Storage 4	Sensitive record recovered from shared external storage.	STORAGE
7	Input Validation 1	SQL injection returned all seeded user records.	CODE
8	Input Validation 2	WebView rendered attacker-controlled `data:` HTML.	PLATFORM
9	Access Control 1	Exported activity opened an API credential screen.	PLATFORM
10	Access Control 2	Intent extra bypassed the PIN workflow.	PLATFORM
11	Access Control 3	Exported provider disclosed private notes.	PLATFORM
12	Hardcoding 2	Native secret extracted; JNI access check observed with Frida.	CRYPTO

CORE CONCLUSION

The application repeatedly trusted client-side storage and client-controlled input. Secrets could be extracted from code, sensitive data could be recovered at rest, and access checks could be bypassed by calling components directly or manipulating Intent extras.

STORAGE AND HARDCODED SECRETS

01 SENSITIVE LOGGING

The checkout flow logged a submitted payment-card value under the `diva-log` tag.

```
E diva-log: Error while processing transaction with credit card: [REDACTED TRAINING CARD]
```

Fix: Do not log sensitive values. Use structured redaction and disable verbose diagnostics in release builds.

02 JAVA HARDCODING

`HardcodeActivity.access()` compared user input to a string embedded directly in the client. The recovered training key granted access.

```
Static review: input.equals("[REDACTED TRAINING KEY]")  
Dynamic confirmation: DIVA displayed "Access granted!"
```

Fix: Keep long-lived secrets and authorization decisions off the client; validate on a server using short-lived, scoped credentials.

03-06 PLAINTEXT STORAGE

Credentials were recovered from four locations. App-private mode reduces routine cross-app access but does not protect plaintext after root compromise, backup exposure, or device extraction. External storage widened exposure.

```
SharedPreferences: /data/data/jakhar.aseem.diva/shared_prefs/...xml  
SQLite: /data/data/jakhar.aseem.diva/databases/ids2  
Private file: /data/data/jakhar.aseem.diva/uinfo*tmp  
External storage: /sdcard/.uinfo.txt
```

```
SQLite validation: SELECT rowid, user, password FROM myuser;
```

Fix: Avoid persisting passwords. Minimize retention, use app-private storage, and protect necessary secrets with Android Keystore-backed cryptography. Never place secrets on shared storage.

INPUT VALIDATION FAILURES

07 SQL INJECTION

The search workflow constructed a query from user input. A single crafted predicate returned all three seeded records, including password and payment fields.

```
Input: ' OR '1'='1' --  
Observed result: all 3 seeded user records returned
```

Fix: Use parameterized queries or Android selection arguments. Minimize returned data and do not persist unnecessary sensitive fields.

08 UNSAFE URI / WEBVIEW HANDLING

The WebView rendered supplied HTML from a `data:` URI. No scheme or trusted-host allow-list protected the input.

```
Input URI: data:text/html,<h1>DIVA_URI_TEST</h1>  
Observed result: WebView rendered DIVA_URI_TEST
```

Fix: Parse with `Uri`, allow only required HTTPS hosts, disable JavaScript unless needed, and avoid privileged bridges to untrusted content.

WHY THIS MATTERS

Input validation is a boundary control. SQL injection altered query logic, while unsafe URI acceptance gave untrusted content a rendering surface.

BROKEN ANDROID COMPONENT ACCESS CONTROL

09 EXPORTED CREDENTIAL ACTIVITY

An unprivileged Android shell caller invoked the implicit activity action directly, bypassing normal application navigation and opening a credential screen.

```
Caller: Android shell UID (non-root)
Action: jakhar.aseem.diva.action.VIEW_CREDS
Observed result: API credential screen opened
```

Fix: Mark sensitive activities non-exported. If inter-app access is necessary, enforce signature permissions and authorize the caller in the receiving component.

10 INTENT EXTRA BYPASS

The credential activity trusted the attacker-controlled `check\_pin` extra. Passing `false` skipped the normal PIN flow.

```
Action: jakhar.aseem.diva.action.VIEW_CREDS2
Extra: check_pin=false
Observed result: protected credential screen opened
```

Fix: Treat every Intent extra as untrusted. Determine authorization from trusted state and enforce it inside the target component.

11 EXPORTED NOTESPROVIDER

The UI PIN protected only the in-app route. The exported ContentProvider allowed a shell caller to read every note directly.

```
content query --uri content://jakhar.aseem.diva.provider.notesprovider/notes
Observed result: 6 private notes returned
```

Fix: Keep providers non-exported by default. Apply signature-level permissions, validate callers, and restrict URI operations.

FRIDA REPAIR AND NATIVE ANALYSIS

INITIAL CONDITION

Frida 17.7.3 process enumeration worked, but every attach or spawn session closed. The target was a rooted Android 11 x86_64 Genymotion image and SELinux was permissive.

ROOT CAUSE

Android Logcat recorded the Frida server aborting during injection with **Unsupported Android linker**. This was an emulator/runtime compatibility issue, not an application defense.

Resolution: a version-matched Frida 16.6.6 Python environment and Android x86_64 server were isolated from the broken Frida 17 setup. Attach, script load, and `Java.perform` then succeeded.

```
Frida 16 attach and script load succeeded.
java-bridge-ok

[+] Hook loaded. Enter the native secret and tap the DIVA button.
[+] DivaJni.access candidate: [REDACTED TRAINING VALUE]
[+] DivaJni.access return value: 1
```

12 NATIVE HARDCODED SECRET

The Java activity passed user input to `DivaJni.access(String)`. Native string extraction identified the candidate value, the app accepted it, and Frida proved the plaintext argument reached the JNI method and returned success.

```
strings -a resources/lib/x86_64/libdivajni.so
...
Java_jakhar_aseem_diva_DivaJni_access
[REDACTED TRAINING VALUE]
```

Fix: Native code can raise reverse-engineering cost but cannot provide durable secret storage. Move sensitive validation to a trusted server.

PRIORITY REMEDIATION PLAN

PRIORITY	ACTION	ADDRESSES
1	Remove long-lived secrets, credential checks, and sensitive business decisions from the client.	Hardcoding 1 and 2; PIN bypass
2	Make sensitive Android components non-exported. Use signature permissions and caller validation only where sharing is required.	Activities and NotesProvider
3	Replace SQL concatenation with parameterized statements / selection arguments.	SQL injection
4	Eliminate password retention; encrypt necessary sensitive values with Android Keystore-backed keys.	Storage 1–4
5	Apply URI scheme and host allow-lists before opening WebView content.	Unsafe URI handling
6	Remove sensitive logging and implement release-safe logging standards.	Insecure logging

KEY LESSON

Mobile apps operate in attacker-controlled environments. Client-side controls can support security, but they cannot be the final authority for secrets or authorization. Secure designs assume the package can be inspected and the process can be instrumented.

DELIVERABLES

Repository: github.com/Calm-Ay/diva-android-security-assessment

Case study: calm-ay.github.io/diva-android-security-assessment/

Portfolio: calm-ay.github.io/